

C*: A Coverage Path Planning Algorithm for Unknown Environments using Rapidly Covering Graphs

Zongyuan Shen[†] James P. Wilson[†] Shalabh Gupta^{†*}

Abstract—The paper presents a novel sample-based algorithm, called C*, for real-time coverage path planning (CPP) of unknown environments. The C* algorithm is built upon the concept of a Rapidly Covering Graph (RCG), which tracks the coverage progress and generates the coverage trajectory in uncovered areas. The RCG is constructed incrementally via progressive sampling of the search space as the environment is discovered during robot navigation. The RCG has a sparse-graph structure formed by efficient sampling and pruning techniques; its nodes and edges form the potential waypoints and segments of the robot's coverage trajectory, respectively. Due to efficient sparse-structure, the RCG produces non-myopic intermediate waypoints of the coverage trajectory. It also guides the robot to escape from the dead-end situations via retreating to the nearest unexplored nodes. A distinct feature of C* is that of progressive sampling which eliminates the need for cellular decomposition of the search space. While C* produces the desired back-and-forth coverage pattern, it adapts to the TSP-based locally optimal coverage of small uncovered regions, called coverage holes, that are surrounded by obstacles and covered regions. Thus, C* proactively detects and covers the coverage holes *in situ*, which reduces the coverage time by preventing the longer return trajectories from distant regions to cover such holes later.

The algorithmic simplicity and low computational complexity of C* makes it easy to implement and suitable for real-time on-board applications. It is analytically proven that C* provides complete coverage of unknown environments. The performance of C* is validated by 1) extensive high-fidelity simulations and 2) real laboratory experiments using autonomous robots. A comparative evaluation with seven existing CPP methods demonstrate that C* yields significant performance improvements in terms of coverage time, number of turns, trajectory length and overlap ratio, while preventing the formation of coverage holes. Finally, C* is evaluated on two different applications of CPP using 1) energy-constrained robots and 2) multi-robot teams. A comparative evaluation of C* with existing methods on these two applications show similar trends where C* trajectories are shorter and with significantly less coverage times.

Index Terms—Coverage path planning, Autonomous robots, Unknown environments, Rapidly covering graph

I. INTRODUCTION

RECENT advancements in sensing, computing and communication technologies have spurred a rapid growth of autonomous service robots that support humans by performing a variety of tasks to improve their quality of life. Specifically, some of the tasks of these robots require covering of all points in the search space. This problem is known as *Coverage Path Planning* (CPP) [1]–[4]. A wide range of application domains exist for CPP including exploration (e.g., mapping of

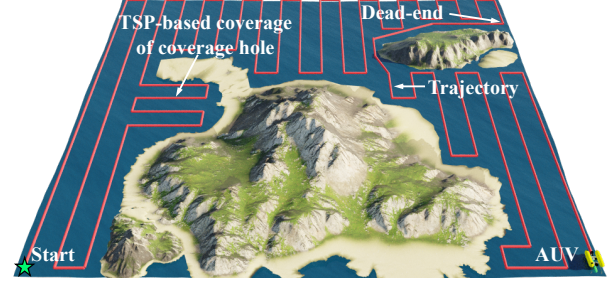


Fig. 1. C*-coverage of an island scenario by an AUV.

underwater terrains [5], [6]); industry (e.g., structural inspection [7]–[10], spray-painting [11], [12] and surface cleaning of vehicles [13]); agriculture (e.g., robotic weeding [14] and arable farming [15]); household (e.g., floor cleaning [16] and lawn mowing [17], [18]); hazard removal (e.g., oil spill cleaning [19], [20]); and defense (e.g., demining [21]).

Often, the environments where coverage operations are performed are unknown. Therefore, it is essential to develop sensor-based CPP methods that can replan and adapt the coverage trajectory in real-time based on dynamic discovery of the environment. It is critical that the CPP method provides complete coverage and does not get stuck in a dead-end. Furthermore, it is desired that the CPP method is efficient, i.e., it minimizes the coverage time by reducing a) the overlaps which in turn reduce the trajectory length, and b) the number of turns. Moreover, it is desired that the coverage method does not create intermediate coverage holes during navigation, which are local uncovered regions surrounded by obstacles and covered regions. This is required to prevent unnecessary inconvenience to cover such holes later via long return trajectories. Finally, it is important that the CPP method is simple to implement on real robot platforms and also computationally efficient to generate adaptive coverage trajectories in real-time.

In this regard, the paper presents a novel algorithm, called C*, for real-time CPP of unknown environments. Fig. 1 shows an example of C*-coverage of an island scenario in an ocean environment by an autonomous underwater vehicle (AUV). While the existing CPP methods rely on different techniques such as cellular decomposition [2], area tiling [3], spanning trees [27], and neural networks [23], C* is a sample-based algorithm built upon a novel concept of rapidly covering graphs (RCGs). An RCG is a compact graphical representation of the obstacle-free space, which is incrementally constructed via progressive sampling during robot navigation. The nodes and edges of RCG form the potential waypoints and segments of the robot's coverage trajectory, respectively. The objectives of RCG are to track the coverage progress, select non-myopic

[†]Dept. of Electrical and Computer Engineering, University of Connecticut, Storrs, CT 06269, USA.

*Corresponding author (email: shalabh.gupta@uconn.edu)

Table I: Comparison of salient features of C* with the baseline algorithms.

	C*	PPCPP [13] (2019)	ε^* [3] (2018)	BA* [22] (2013)	NNCPP [23] (2008)	BM [24] (2007)	BSA [25] (2005)	FSSTC [26] (2003)
Environment Representation	RCG	Circular disks	Grid map	Grid map	Grid map	Grid map	Grid map	Grid map
Coverage Pattern	Back and forth with adaptive TSP-optimized coverage of coverage holes	Spiral	Back and forth	Back and forth	Back and forth	No obvious pattern observed	Spiral	Spiral
Coverage Method	Creates RCG incrementally by progressive sampling, RCG tracks coverage and generates waypoints for navigation	A reward function guides waypoint generation in the neighborhood	ETM generates the navigation and tasking commands using the tracking information stored in MAPS	Follows a priority of north-south-east-west to pick waypoints	Dynamic neural activity landscape guides waypoint generation	Picks unexplored cell in the neighborhood as waypoint to gradually thicken the blocks of inaccessible cells	Follows covered area or obstacles on a specific lateral side	Spanning tree guides waypoint generation in the neighborhood
Waypoint Selection	Non-myopic due to sparse structure of RCG	Myopic	Myopic	Myopic	Myopic	Myopic	Myopic	Myopic
Proactive Coverage Holes Prevention	Detects coverage holes during back & forth motion and covers them <i>in situ</i> by a TSP path	None	None	None	None	None	None	None
Escape from Dead-end	Moves to the nearest retreat node	Moves to the nearest uncovered point	Searches MAPS for high-potential uncovered regions	Moves to the nearest back-tracking point	Moves to uncovered area using neural activity landscape	Picks an explored cell repeatedly until an unexplored cell is found	Moves to the nearest back-tracking point	Moves to parent cell until a new uncovered neighbor is found

waypoints for the coverage trajectory in uncovered areas and to guide the robot in escaping from the dead-end situations.

C* provides complete coverage guarantee of unknown environments using the desired back and forth coverage pattern. During navigation, if the robot detects any coverage holes, which are uncovered regions surrounded by obstacles and covered regions, then C* synergistically adapts to the local optimal coverage of such coverage holes by the trajectory generated *in situ* using the traveling salesman problem (TSP). This eliminates the need to cover such holes later by the longer return trajectories from distant regions. In comparative evaluation with the existing CPP methods, it is shown that C* is very efficient and yields significant performance improvements in terms of coverage time, number of turns, path length and overlap ratio. Furthermore, C* is simple to implement on real robots and computationally efficient for real-time operations.

A. Related Work

A review of existing CPP methods is presented in [4]. In general, the CPP methods are classified as random or systematic. While the random methods are simple to implement, they can generate strongly overlapping trajectories and do not guarantee complete coverage in finite time. In contrast, the systematic methods use some basic underlying rules to generate well-defined coverage patterns (e.g., back & forth and spiral) and provide complete coverage in finite time.

The CPP methods are also classified based on the dimension of coverage space. While the 2D methods plan coverage paths to cover the 2D surfaces (e.g., floor cleaning), the 3D methods work in the 3D spaces (e.g., mapping of high-rise buildings by UAVs and underwater terrains by UUVs). Hert et. al. [28] presented a CPP method for underwater terrain mapping using an AUV by sweeping across the terrain while moving at a

fixed distance to the uneven surface. Shen and Gupta [6] [29] presented an algorithm, called CT-CPP, that performs layered sensing of the environment using coverage trees (CT) for terrain mapping.

Finally, the CPP methods are classified as offline or online depending on the availability of the environmental information. The offline methods rely on prior information known about the environment to design the coverage paths before the robotic system is deployed for operation. Although these algorithms perform well in known scenarios, their performance can degrade if the prior information is incomplete and/or incorrect. In contrast, the online methods are adaptive [30], i.e., they update the coverage paths in real-time based on the environmental information collected by the onboard sensors.

Several CPP methods have been developed for known environments. Zelinsky et al. [31] proposed a grid-based method, which assigns each grid cell a distance-to-goal value. The coverage path is then generated along the steepest ascent by selecting the unvisited cell in the local neighborhood with the largest distance-to-goal value. Huang [32] developed a CPP method that divided the coverage area into multiple subareas which are each covered by back-and-forth motion with different sweep directions to minimize the total number of turns. Xie et al. [33] proposed a TSP-based method to cover multiple disjoint areas for UAVs. Ramesh et al. [34] computed the optimal partitioning of a non-convex environment for minimizing the number of turns by solving a linear programming problem. Shen et al. [35] presented a neural network based method, called Coverage Path Planning Network (CPPNet), which takes the occupancy grid map of the environment as input and generates a near-optimal coverage path offline.

Several CPP methods have been developed for unknown environments. Lee et al. [36] proposed a grid-based method

using a path linking strategy called coarse-to-fine constrained inverse distance transform (CFCIDT) to generate a smooth coverage path, so as to avoid sharp turns. Gabriely and Rimon [27] developed the Spanning Tree Covering (STC) algorithm to generate a systematic spiral path by following a spanning tree. Later in [26], this method was improved to full spiral STC (FS-STC) algorithm. Gonzalez et al. [25] presented the Backtracking Spiral Algorithm (BSA), which generates a spiral path online by following the covered area or obstacles on a specific lateral side until no uncovered free cell can be found in the local neighborhood. Then, a backtracking mechanism is used for escaping from the dead-end situation.

Acar and Choset [2] developed a CPP method which detects the critical points on obstacles to decompose the coverage area into multiple subareas. Then, these subareas and their connections are represented by a graph and coverage is achieved by covering each subarea individually. A limitation of this method is that it can not handle rectilinear environments. This method was further improved by Garcia and Santos [37]. Ferranti et al. [24] presented the Brick-and-Mortar (BM) algorithm, which gradually enlarges the blocks of inaccessible cells (i.e., visited or obstacle cells) by selecting the target cell as the unexplored cell in the local neighborhood with the most surrounding inaccessible cells. The target cell is marked as visited if it does not block the path between any two accessible cells (i.e., explored or unexplored cells) in the neighborhood; otherwise, it is marked as explored. If there is no unexplored cell in the neighborhood, then an explored cell is selected to traverse through and escape from the dead-end situation.

Luo and Yang [23] developed the Neural Network CPP (NNCPP) algorithm, which places a neuron at the center of every grid cell. The robot path is planned online based on the dynamic activity landscape of the neural network. The activity landscape is updated such that the obstacle cells have negative potential values for collision avoidance and uncovered cells have positive potential values for coverage. The algorithm can escape from the dead-ends by means of neural activity propagation from the unclean areas to attract the robot. Gupta et al. [38] proposed an Ising model approach for dynamic CPP. Viet et al. [22] presented the Boustrophedon motions and A* (BA*) algorithm, which performs the back-and-forth motion until no uncovered cell is found in the local neighborhood. Then, to escape from a dead-end situation, the algorithm searches for and computes the shortest path to the nearest backtracking cell using the A* algorithm [39].

Hassan and Liu [13] proposed the Predator-Prey CPP (PPCPP) algorithm, which is based on a reward function consisting of three factors: 1) predation avoidance reward that maximizes the distance from the predator to prey (predator is a virtual stationary point and prey is the coverage spot of the robot), 2) smoothness reward that minimizes the turn angle for continuing motion in a straight line, and 3) boundary reward for covering boundary targets. Song and Gupta [3] developed the ε^* algorithm, which utilizes an Exploratory Turing Machine (ETM) as a supervisor to guide the robot online with navigation and tasking commands for coverage via back-and-forth motion. To escape from a dead-end, the algorithm uses multi-resolution adaptive potential surfaces

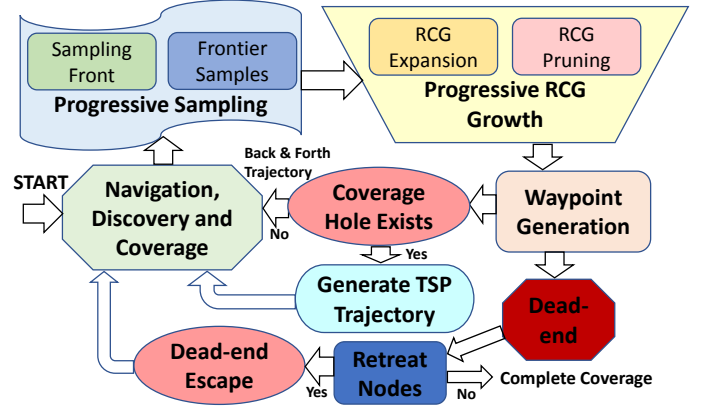


Fig. 2. C* Operation.

(MAPS) to find the unexplored regions.

Recently, some CPP methods have been proposed considering the energy constraints of robots. Shen et al. [40] proposed the ε^*+ algorithm which is an extension of the ε^* algorithm [3] for energy-constrained vehicles. It generates the back-and-forth coverage path while continuously monitoring the remaining energy of the robot if it is sufficient to return back to the charging station. Upon battery depletion, the robot retreats back to the charging station using the A* algorithm and after recharging it advances to the nearest unexplored cell to restart the coverage of the remaining uncovered area. Shnaps and Rimon [41] developed the battery powered coverage (BPC) algorithm, where the robot follows circular arcs formed by equidistant points to the source. When the battery discharges to a level just sufficient to return to the base, the robot returns to the charging station along the shortest path. Dogru and Marques [42] proposed energy constrained online path planning (ECOCPP) algorithm, which generates coverage and retreat paths by following the contours of covered area.

Some online CPP methods have also been proposed for multi-robot systems [43]–[46] and curvature-constrained robots [14], [47]–[49].

B. Summary of the C* Algorithm

This section provides a brief summary of the C* algorithm. C* provides complete coverage of unknown environments with dynamic sensing and adaptive coverage trajectory generation. It produces the desired back-and-forth coverage pattern with *in situ* adaptation to the TSP-optimized coverage of coverage holes that are detected along the path during navigation. While most CPP methods are grid-based, C* is sample-based. It is built upon a novel concept of RCG which is incrementally built as the robot discovers the environment.

Fig. 2 illustrates the C* operation. While traversing towards a waypoint on the coverage trajectory, the robot senses and discovers the environment. It then incrementally grows the RCG by progressive sampling in the obstacle-free region detected by onboard sensors (details in Section III-B1). The progressive sampling is done in a systematic manner within the sampling front using the frontier samples that are adjacent to the unknown area or obstacles (details in Section III-B2). Then, the nodes of RCG are formed by these samples and

serve as the waypoints for the robot. On the other hand, the edges are formed by connecting the nodes and serve as the potential traversal paths for the robot to form the coverage trajectory. The RCG forms a compact graphical representation of the observed obstacle-free space (details in Section III-B3). It has a sparse structure built by efficient sampling and pruning strategies, which provides computational and memory-efficient operation and non-myopic waypoint generation. To track the coverage progress, the nodes of RCG maintain the state as closed (visited) or open (unvisited). The next navigation waypoint of the robot is selected as an unvisited neighboring node on the RCG. The waypoint is selected in a manner to generate the desired back-and-forth coverage pattern (details in Section III-B4). Once the waypoint is selected, the robot moves towards it, maps the environment and covers the region around the trajectory, and the above process is repeated.

1) **Escaping Dead-ends:** C^* provides an effective strategy for escaping from dead-ends. It is possible that the robot reaches a point where there is no unvisited node on the RCG in the local neighborhood, known as a dead-end. To escape from the dead-ends, C^* utilizes a dead-end escape strategy, which is built upon a concept of retreat nodes. The retreat nodes are all unvisited nodes on the RCG which are adjacent to the visited nodes. The dead-end escape strategy searches for the nearest retreat node using the A* algorithm [39] and sets it as the next waypoint. Upon reaching this node, the robot resumes the back-and-forth coverage of the remaining area.

2) **Prevention of Coverage Holes:** C^* provides a proactive strategy for preventing the formation of coverage holes.

Definition 1.1 (Coverage Hole). *An obstacle-free uncovered region is said to be a coverage hole if it is surrounded by a) obstacles and b) covered regions.*

By following a certain coverage pattern (e.g., back-and-forth or spiral), it is possible that the robot sometimes bypasses uncovered secluded regions that are surrounded by obstacles and covered regions. If left unattended, these regions form coverage holes, as shown in Fig. 3a. Most CPP methods cover such holes later by return trajectories from distant regions, thus causing unnecessary overlaps, longer trajectory lengths and higher coverage times. To solve this issue, C^* detects such secluded regions *in situ* and covers them using TSP-based locally optimal coverage trajectories before resuming the back and forth search to cover the remaining area, as shown in Fig. 3b. This proactive strategy prevents the formation of coverage holes, thereby improves the coverage performance and provides user satisfaction and convenience.

C. Comparison of C^* with the Baseline Algorithms

Table I presents the salient features of C^* in comparison to seven different baseline algorithms. First, the baseline algorithms are grid-based, but C^* is sample-based. While the baseline algorithms maintain a fine grid map, C^* , on the other hand, uses the RCG to represent the environment and monitor the coverage progress. The RCG is incrementally built using progressive sampling of the observed area. Second, the waypoint generation of baseline algorithms is myopic,

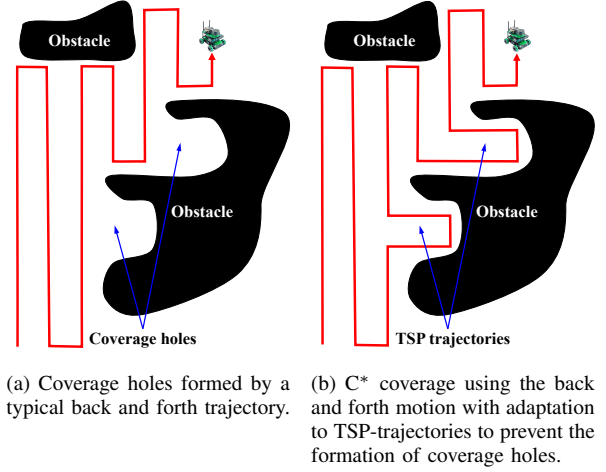


Fig. 3. Coverage holes prevention by C^* using TSP-trajectories.

i.e., within the local neighborhood on a grid. On the other hand, the RCG is built with a sparse-graph structure using efficient sampling and pruning techniques; thus, it provides high computational and memory efficiency with non-myopic waypoint generation. Third, the baseline algorithms could produce coverage holes, which are covered by return trajectories, thus causing strongly overlapped trajectories and high coverage times. In contrast, C^* actively detects potential coverage holes during navigation, performs TSP-based optimal coverage of such regions, and then resumes the back-and-forth motion. Finally, the baseline algorithms that yield spiral paths are limited in their application when turning is regarded expensive; in contrast, C^* produces the desired back-and-forth coverage pattern. Furthermore, in comparison to the cellular decomposition methods [2], C^* does not require to detect the critical points on the obstacles. Moreover, the algorithmic simplicity and computational efficiency of C^* make it easy to implement on robotic processors for real-time applications.

D. Contributions

The main contributions of this paper are summarized below.

- Development of a novel CPP algorithm, called C^* , for real-time adaptive coverage of unknown environments, with the following features:
 - Progressive sampling to create frontier samples during navigation as the environment is discovered.
 - Incremental construction of the RCG using the frontier samples, to track the coverage progress and plan the coverage trajectory using non-myopic waypoints.
 - Escape from the dead-ends using the concept of retreat nodes.
 - Prevention of coverage holes by using TSP-based local trajectory optimization.
- Comparative evaluation of the performance of C^* with the existing methods using high-fidelity simulations.
- Testing and validation of C^* by real experiments on autonomous robots in a laboratory setting.
- Comparative evaluation of the performance of C^* on two different CPP applications using 1) energy-constrained robots and 2) multi-robot teams.

E. Organization

The remaining paper is organized as follows. Section II presents the CPP problem and the objectives of C^* . Section III describes the details of the C^* algorithm with its iterative steps. Section IV provides the algorithm complexity and complete coverage proof. Section V shows the comparative evaluation results of C^* with the baseline algorithms along with the experimental validation. This section also presents the application and comparative evaluation of C^* for energy-constrained CPP and multi-robot CPP problems. Finally, Section VI concludes the paper with suggested recommendations for future work.

II. PROBLEM DESCRIPTION

Let $\mathcal{R}$ be a coverage robot, as shown in Fig. 4a, which is equipped with the following:

- A localization device (e.g., GPS, IMU, wheel encoder, and an Indoor Localization System) or SLAM [18] to locate the position of the robot,
- A range detector and mapping sensor (e.g., lidar and ultrasonics) of range $r_d \in \mathbb{R}^+$, to detect obstacles within its field of view (FOV) and map the environment, and
- A coverage device/sensor (e.g., cleaning brush) of range $r_c \in \mathbb{R}^+$, to carry out the main coverage task.

Let $\mathcal{A} \subset \mathbb{R}^2$ be the unknown area populated by obstacles of arbitrary shapes that is tasked for coverage by robot $\mathcal{R}$. Let $\mathcal{A}_o \subset \mathcal{A}$ be the space occupied by obstacles. Then the obstacle-free space $\mathcal{A}_f = \mathcal{A} \setminus \mathcal{A}_o$ is the coverage space that is desired to be covered by the coverage device of robot $\mathcal{R}$ to perform a certain task (e.g., cleaning and demining). It is assumed that the coverage space is connected, i.e., any point in the coverage space is reachable by the robot from any other point, to form the complete coverage trajectory.

Definition II.1 (Coverage Trajectory). *Let $\lambda : [0, T] \rightarrow \mathcal{A}_f$, where $\lambda(t) \in \mathcal{A}_f$ is the point visited by robot $\mathcal{R}$ at time $t \in [0, T]$ and $T \in \mathbb{R}^+$ is the total coverage time. Then, the coverage trajectory is defined as $\Lambda_T \triangleq [\lambda(t)]_{t=0}^T$, with $\lambda(0)$ and $\lambda(T)$ as its start and end points, respectively.*

Fig. 4b shows an example of the coverage trajectory and the area covered by the robot's coverage device during navigation. The coverage trajectory has to be generated such that the robot's coverage device covers the coverage space $\mathcal{A}_f$.

Definition II.2 (Complete Coverage). *Let $\mathcal{A}_c(\lambda(t)) \subset \mathcal{A}_f$ be the area covered by the coverage device of robot $\mathcal{R}$ at point $\lambda(t) \in \Lambda_T$ on the coverage trajectory. Then, the coverage of $\mathcal{A}_f$ is said to be complete if*

$$\mathcal{A}_f \subseteq \bigcup_{t \in [0, T]} \mathcal{A}_c(\lambda(t)). \quad (1)$$

The objective of C^* is to generate the coverage trajectory Λ_T of robot $\mathcal{R}$ to provide complete coverage of $\mathcal{A}_f$. It is essential that C^* does not get stuck in dead-end situations and provide uninterrupted coverage. Furthermore, it is envisioned that C^* produces the desired back and forth coverage pattern with synergistic adaptations to the TSP-based locally optimal coverage trajectories to prevent the formation coverage holes.

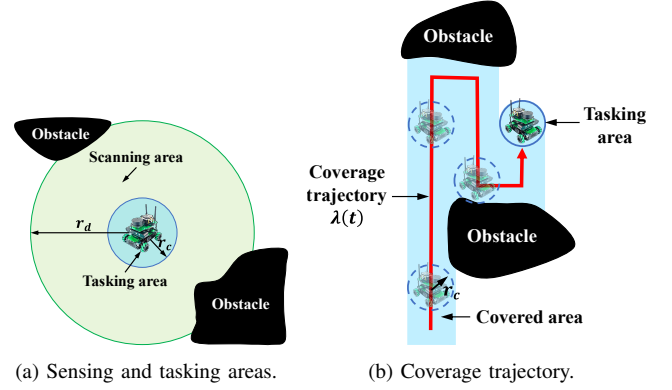


Fig. 4. Robot: a) sensing and tasking areas and b) coverage trajectory.

Note that significant coverage time could be saved by preventing coverage holes to avoid return trajectories from distant regions to cover such holes later. Moreover, it is desired that C^* minimizes the coverage time by minimizing the trajectory length, overlaps, and the number of turns.

III. C^* ALGORITHM

C^* is an adaptive CPP algorithm for unknown environments. It generates the coverage trajectory incrementally during robot navigation based on the environment information collected by sensors. C^* generates the coverage trajectory via the following iterative steps until complete coverage is achieved.

1. *Navigation, discovery and coverage*: In this step, the robot moves towards the current waypoint of the coverage trajectory, discovers and maps the environment using its onboard sensors (e.g., the range detector) and in parallel performs the coverage task using its coverage device.
2. *Progressive sampling*: Once the robot reaches the current waypoint, this step creates a sampling front in the newly discovered area and systematically generates new samples for the purpose of growing the RCG.
3. *Progressive RCG growth*: Once the sampling is done, this step grows the current RCG using the new samples via efficient expansion and pruning strategies.
4. *Coverage trajectory generation*: Once the RCG is updated, this step determines the next waypoint of the coverage trajectory using the updated RCG.

A. Overview of the Algorithm Steps

Let the iterations of C^* be denoted by $i \in \{0, 1 \dots m\}$, where $m \in \mathbb{N}^+$. The total number of iterations is $m + 1$ in which complete coverage is achieved. In each iteration i the above four steps are performed. Let $p_0 \in \mathcal{A}_f$ be the starting point.

At iteration $i = 0$, C^* initializes when the robot arrives at p_0 at time $t_0 = 0$, i.e., $\lambda(t_0) = p_0$. Then, it stays at p_0 and senses the environment within the FOV of its range detector. Next, it samples the discovered area and generates the first installment of samples. (Details of sampling are in Section III-B2). Using these samples, it creates the first portion of RCG by setting samples as nodes and connecting nodes to form edges. (Details of RCG construction are in Section III-B3). Then, this first portion of RCG, in turn, selects a node as an intermediate goal

and sets its position as the first waypoint $p_1 \in \mathcal{A}_f$. (Details of waypoint generation are in Section III-B4). This finishes iteration $i = 0$ with the output to the robot as waypoint p_1 .

Next, at iteration $i = 1$, the robot moves from p_0 to p_1 , thus forming the first portion of coverage trajectory. During this motion, the robot continuously discovers the environment using its range detector to incrementally build the map. While navigating the robot also performs the coverage task along the trajectory using its coverage device/sensor. The robot reaches p_1 at time $t_1 \in (0, T]$, i.e., $\lambda(t_1) = p_1$. Upon reaching p_1 , it repeats Steps 2 – 4 of sampling, RCG growth and trajectory generation. This finishes iteration $i = 1$ with the output to the robot as the next waypoint $p_2 \in \mathcal{A}_f$.

In general, at iteration $i \in \{1, \dots, m-1\}$ the robot starts at point $p_{i-1} \in \mathcal{A}_f$. The input to iteration i is the next waypoint $p_i \in \mathcal{A}_f$, which was generated as the output of iteration $i-1$. Thus, at iteration i , the robot moves from p_{i-1} to p_i and forms the i^{th} portion of the coverage trajectory. During this motion it discovers the environment using its range detector. It also performs the coverage task along the coverage trajectory using its coverage device/sensor. The robot reaches p_i at time $t_i \in (0, T]$, i.e., $\lambda(t_i) = p_i$. Upon reaching p_i , it generates samples in the newly discovered area. Note that the newly discovered area is all the area that has been sensed during motion from p_{i-1} to p_i and that has not been previously observed. Once the samples are created, it grows the existing RCG using these samples. Finally, the updated RCG generates the next waypoint $p_{i+1} \in \mathcal{A}_f$ to continue forming the coverage trajectory until complete coverage is achieved. Note that it is possible that at a certain iteration no new area is discovered depending on the scenario; thus, no new samples are created and RCG is not grown. In this case, the RCG relies on its existing structure and uses its available nodes to generate the next waypoint.

At iteration $m-1$, the RCG generates the final destination point $p_m \in \mathcal{A}_f$. This means that at the end of iteration $m-1$, the RCG is a) fully grown, i.e., the coverage area is fully discovered, sampled, and no more eligible samples exist that could be used to grow the RCG, and b) fully exploited, i.e., the last available node is selected to generate p_m as the final waypoint on the coverage trajectory. At the final iteration m , the robot simply starts at p_{m-1} and moves to p_m . At this stage, no new area is discovered; thus, no new frontier samples are created and the RCG is not grown any further. Thus, upon reaching the waypoint p_m , the coverage is complete.

In this manner, C^* generates the waypoint sequence to form the coverage trajectory as described below.

Definition III.1 (Waypoint Sequence). *The waypoint sequence $\mathcal{P} = \{p_0, p_1, \dots, p_m\}$ is the sequence of intermediate goal points $p_i \in \mathcal{A}_f$ for the robot $\mathcal{R}$ to form the coverage trajectory, where p_0 and p_m are its start and end points, respectively.*

By following the waypoint sequence $\mathcal{P}$, the robot forms the coverage trajectory $\Lambda_T \equiv \lambda(t_0) \rightarrow \lambda(t_1) \rightarrow \dots \lambda(t_m)$, such that $\lambda(t_i) = p_i, i = 0, \dots, m$, where $t_0 = 0$ and $t_m = T$. Note that $p_i \neq p_j, \forall i \neq j, i, j \in \{0, 1, \dots, m\}$ to avoid any loops.

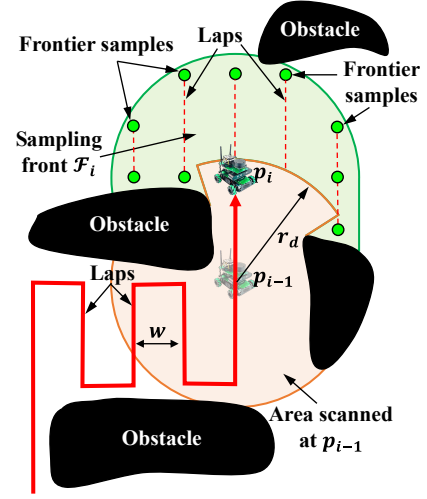


Fig. 5. An example showing i) dynamic discovery of the environment during navigation from p_{i-1} to p_i to generate the sampling front $\mathcal{F}_i$, and ii) progressive sampling on $\mathcal{F}_i$ to generate the frontier samples along the boundary of unknown and obstacle regions.

B. Details of the Algorithm Steps

The details of the four steps are presented below.

1) Navigation, Discovery and Coverage: Since C^* operates in unknown environments, it first collects data using onboard sensors to dynamically discover and map the environment in order to find a coverage trajectory. In each iteration it could only construct a partial map of the environment.

At iteration $i = 0$, the robot arrives at the start point p_0 , where it senses the environment to make an initial map around p_0 . Subsequently, at iteration $i \in \{1, \dots, m-1\}$, the robot has a waypoint p_i available. As such, it moves from $\lambda(t_{i-1}) = p_{i-1}$ at time t_{i-1} and reaches $\lambda(t_i) = p_i$ at time t_i , thus forming the i^{th} portion of the coverage trajectory. During this motion from p_{i-1} to p_i , the robot performs the coverage task using its coverage device. It also continuously discovers the environment and maps the obstacles within the FOV of its range detector. Fig. 5 shows the dynamic discovery process.

Let $\mathcal{A}_d(\lambda(t)) \subset \mathcal{A}$ be the area scanned by the range detector when the robot is at a point $\lambda(t) \in \mathcal{A}_f$ at time t . Then, the total area discovered and mapped in the i^{th} iteration is the cumulation of the area scanned from time t_{i-1} to t_i , given as

$$\mathcal{A}_{d,i} = \bigcup_{t \in [t_{i-1}, t_i]} \mathcal{A}_d(\lambda(t)). \quad (2)$$

This area $\mathcal{A}_{d,i}$ is used for progressive sampling and growing the RCG in the next steps. The total area discovered from the beginning of the coverage operation is given as $\mathcal{A}'_d = \bigcup_i \mathcal{A}_{d,i}$. The remaining area $\mathcal{A}_u = \mathcal{A} \setminus \mathcal{A}'_d$ is still unknown.

At iteration m , the robot moves from p_{m-1} to p_m and senses the environment; however, since all area has been already mapped until iteration $m-1$, no new area is discovered.

2) Progressive Sampling: The progressive sampling is the process of incremental sampling in the newly discovered area $\mathcal{A}_{d,i}$. At iteration $i \in \{0, \dots, m-1\}$, once the robot reaches p_i and completes the discovery step, C^* starts the progressive

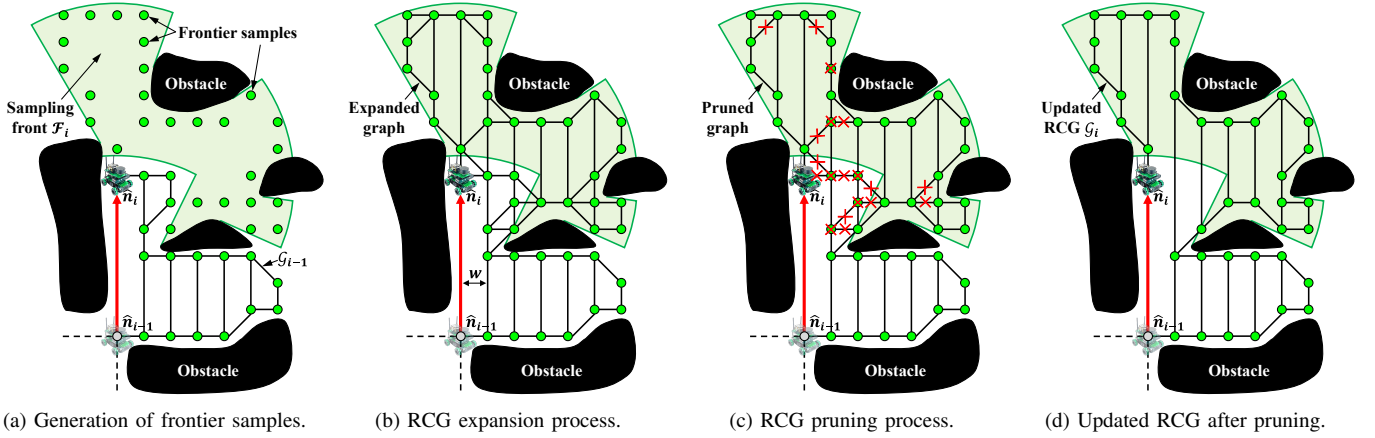


Fig. 6. Growth of RCG at iteration i by 1) expansion into $\mathcal{F}_i$ using the frontier samples and 2) pruning the inessential nodes and edges.

sampling step. The objective of sampling is to grow the RCG in $\mathcal{A}_{d,i}$. The RCG, in turn, generates the new waypoint p_{i+1} to form the future trajectory of the robot. Note that at the final iteration m there is no need for progressive sampling since the robot reaches the end point p_m of the coverage trajectory.

a) Creation of Sampling Front: For progressive sampling, $\mathcal{C}^*$ first creates a sampling front.

Definition III.2 (Sampling Front). *The sampling front $\mathcal{F}_i \subseteq \mathcal{A}_{d,i}$, $i \in \{0, \dots, m-1\}$, is the obstacle-free and unsampled portion of the area $\mathcal{A}_{d,i}$ discovered in the i^{th} iteration.*

Figs. 5 and 6a show examples of the sampling front $\mathcal{F}_i$ generated by the robot during navigation from p_{i-1} to p_i . Note that it is possible that some portion of $\mathcal{A}_{d,i}$ has been already discovered before and sampled in a previous iteration. Thus, $\mathcal{F}_i$ includes only the newly discovered and unsampled area.

b) Sampling Strategy: The sampling strategy is described next. Let $\mathcal{S}$ be the set of all feasible samples that lie in the coverage space $\mathcal{A}_f$. Once $\mathcal{F}_i$ is generated, it is sampled with the frontier samples $\mathcal{S}_i \subseteq \mathcal{S}$ as defined below.

Definition III.3 (Frontier Sample). *A sample $s \in \mathcal{S}$ is said to be a frontier sample if it is adjacent to*

- the unknown area, and/or
- an obstacle or boundary.

Thus, a sample $s \in \mathcal{S}$ is determined to be a frontier sample if a ball $\mathcal{B}(s, w)$ of radius w centered at s contains unknown and/or obstacle area. The set of frontier samples $\mathcal{S}_i \subseteq \mathcal{S}$ is systematically generated in the sampling front $\mathcal{F}_i$ by sampling on laps as described below.

Definition III.4 (Lap). *A lap is a virtual straight line segment in the coverage space whose two ends terminate either at an obstacle or the coverage space boundary.*

The laps are created in the sampling front $\mathcal{F}_i$ for the purpose of efficient, nonuniform and sparse sampling. At iteration $i \in \{0, \dots, m-1\}$, the laps are drawn in the sampling front $\mathcal{F}_i$, such that they are parallel to each other and the distance between any two adjacent laps is equal to the sampling resolution $w \in \mathbb{R}^+$. The laps are created along the direction of desired back

and forth motion. The first lap is drawn in the sampling front $\mathcal{F}_0$ starting from p_0 . Note that it is possible that only portions of laps are created within $\mathcal{F}_i$, that is their ends may not reach an obstacle or the boundary. The laps are also drawn to be consistent, which means that any two laps separated by an obstacle in between, such that one lap is above and the other below the obstacle, are aligned on the same line whenever possible. Fig. 5 shows the examples of laps on the coverage trajectory and the new ones created in $\mathcal{F}_i$ for sampling.

Once the laps are created in $\mathcal{F}_i$, they are sampled with the frontier samples $\mathcal{S}_i \subseteq \mathcal{S}$. The first sample is generated at p_0 in $\mathcal{F}_0$ at iteration $i = 0$. Thereafter, all laps in $\mathcal{F}_0$ are populated with samples. The sampling procedure is as follows. At iteration $i \in \{0, \dots, m-1\}$, the samples are placed on each lap in $\mathcal{F}_i$, such that the spacing between any two adjacent samples on a lap is δw , $\delta \in \mathbb{N}^+$, where $\delta \geq 1$ is the minimum multiplier that allows the placement of a frontier sample. Note that multiple frontier samples could be placed on a lap within a sampling front depending on the obstacle geometries.

The progressive sampling produces nonuniform and sparse samples which provide computational and memory efficiency and enable non-myopic waypoint selection (Section III-B4). Figs. 5 and 6a show examples of progressive sampling.

3) Progressive RCG Growth: The objectives of a Rapidly Covering Graph (RCG) are to track the coverage progress, generate adaptive waypoints to produce the coverage trajectory through unexplored regions, and provide escape routes from the dead-ends. First, we define an RCG as follows.

Definition III.5 (RCG). *An RCG $\mathcal{G} = (\mathcal{N}, \mathcal{E})$ is defined as a simple, connected, and planar graph, where*

- $\mathcal{N} = \{n_j : j = 1, \dots, |\mathcal{N}|\}$ is the node set, such that each node corresponds to a frontier sample and represents a waypoint on the coverage trajectory of the robot, and
- $\mathcal{E} = \{e(n_j, n_k) : j = 1, \dots, |\mathcal{N}|, k = 1, \dots, |\mathcal{N}|\}$ is the edge set, such that each edge connects two nodes and represents a collision-free traversal path for the robot.

Definition III.6 (Neighbors). *A node $n' \in \mathcal{N}$ is said to be a neighbor of a node $n \in \mathcal{N}$ and vice versa if there exists an edge $e(n, n') \in \mathcal{E}$ connecting n' and n .*

Definition III.7 (Neighborhood). *The neighborhood of a node $n \in \mathcal{N}$ is the set $\mathcal{N}_b(n) \subset \mathcal{N}$ of all its neighbors.*

At iteration $i \in \{0, \dots, m-1\}$, the RCG $\mathcal{G}_i = (\mathcal{N}_i, \mathcal{E}_i)$ is grown in the sampling front $\mathcal{F}_i$ using the RCG a) expansion and b) pruning strategies described below. Fig. 6 shows an example of the RCG expansion and pruning processes.

a) RCG Expansion Strategy: The RCG expansion strategy describes the procedure to add new nodes and edges to the existing RCG. Initially, the RCG has empty node and edge sets. Subsequently, at each iteration $i \in \{0, \dots, m-1\}$, once the robot reaches the waypoint p_i and performs the progressive sampling, the RCG is expanded into the sampling front $\mathcal{F}_i$ using the frontier samples. Fig. 6b shows an example of RCG expansion into the sampling front $\mathcal{F}_i$.

The RCG is expanded as follows. First, all the frontier samples $\mathcal{S}_i$ in $\mathcal{F}_i$ are assigned to be the new nodes $\mathcal{N}_{new}$ of RCG, where $|\mathcal{N}_{new}| = |\mathcal{S}_i|$. The new nodes are added to the existing node set $\mathcal{N}_{i-1}$ to get $\mathcal{N}'_{i-1} = \mathcal{N}_{i-1} \cup \mathcal{N}_{new}$. Then, the new edges $\mathcal{E}_{new}$ are created by making valid connections between new nodes and to the nodes of existing RCG. To add the edges all laps in the coverage space $\mathcal{A}_f$ are considered. Note that a lap in $\mathcal{A}_f$ may contain both new and old nodes of the existing RCG. Thus, for each new node $n \in \mathcal{N}_{new}$ on each lap, new edges are drawn to connect it with

- i. the adjacent nodes on the same lap.
- ii. all nodes within a distance of $\sqrt{2}w$ on each adjacent lap.

Note that it is possible that a node has more than two adjacent laps on one side separated by an obstacle. Each edge is checked for feasibility before connecting, i.e., if it lies in $\mathcal{A}_f$. In this manner, all possible edges are drawn from each new node that connect it to: 1) the nodes above and below on the same lap, and 2) the nodes on the left and right adjacent lap(s). The new edges are added to the existing edge set $\mathcal{E}_{i-1}$ to get $\mathcal{E}'_{i-1} = \mathcal{E}_{i-1} \cup \mathcal{E}_{new}$. In this fashion, by adding new nodes and edges, the RCG is expanded at iteration i into the sampling front $\mathcal{F}_i$ to get $\mathcal{G}'_{i-1} = \{\mathcal{N}'_{i-1}, \mathcal{E}'_{i-1}\}$.

b) RCG Pruning Strategy: Once the RCG is expanded into $\mathcal{F}_i$, it is pruned to maintain a sparse graph structure for computational and memory efficiency. This is done by pruning redundant nodes and edges in the expanded graph $\mathcal{G}'_{i-1}$.

Definition III.8 (Essential Node). *A node $n \in \mathcal{N}$ is said to be essential if it is*

1. adjacent to the unknown area, or
2. an end node of a lap, or
3. not an end-node but connected to an end node, say $n_x \in \mathcal{N}$, of an adjacent lap, i.e., $n \in \mathcal{N}_b(n_x)$, and
 - a. n_x has no other neighbor on n 's lap, or
 - b. n_x has other neighbors on n 's lap which are all non-end nodes but the edge $e(n, n_x)$ connecting n and n_x is closest to an obstacle or the unknown area.

The first condition above implies that a node is essential if it can enable future connections to the unknown area. The second condition is important to reach the end of a lap and complete its coverage. The third condition is required to enable

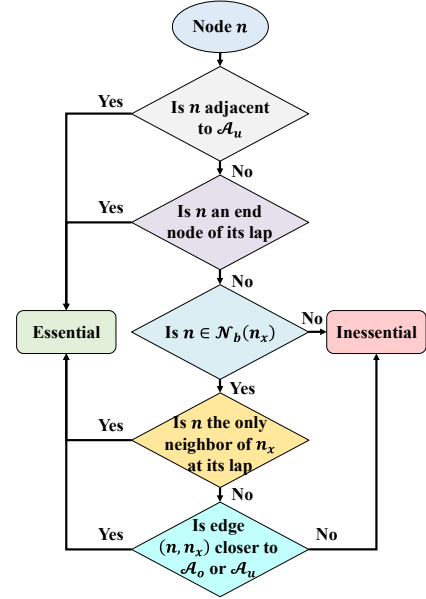


Fig. 7. Flowchart to determine essential and inessential nodes.

transition between laps for connectivity and to make sure that the transition edge is closest to the obstacle for maximum coverage of the obstacle-free area. Fig. 7 shows the flowchart to determine the essential and inessential nodes.

The node pruning is done as follows. Let $\mathcal{N}_{\partial F_i} \subseteq \mathcal{N}_{i-1}$ be the set of previous nodes that are adjacent to the boundary ∂F_i of the sampling front $\mathcal{F}_i$. For node pruning, each node $n \in \{\mathcal{N}_{new} \cup \mathcal{N}_{\partial F_i}\}$ is checked for the conditions of Def. III.8 to find if it is essential or inessential. Then, the set of inessential nodes $\mathcal{N}_{iness}$ is pruned and the remaining nodes are kept in RCG. The next step is to prune the inessential edges.

Definition III.9 (Essential Edge). *An edge $e(n_x, n_y) \in \mathcal{E}$ connecting nodes n_x and n_y is said to be essential if n_x and n_y are both essential nodes and*

1. n_x and n_y are on the same lap, or
2. n_x and n_y are on the adjacent laps, and
 - a. n_x and n_y are the end nodes of laps, or
 - b. n_x is an end-node and n_y is a non-end node, and
 - i. n_x has no other neighbor on n_y 's lap or
 - ii. n_x has other neighbors on n_y 's lap which are all non-end nodes, but the edge $e(n_x, n_y)$ is closest to an obstacle or the unknown area.

The edge pruning is done as follows. Let $\mathcal{E}_{\partial F_i} \subseteq \mathcal{E}_{i-1}$ be the set of previous edges connected to $\mathcal{N}_{\partial F_i}$. For edge pruning, each edge $e \in \{\mathcal{E}_{new} \cup \mathcal{E}_{\partial F_i}\}$ is checked for the conditions of Def. III.9 to find if it is essential or inessential. Then, the set of inessential edges $\mathcal{E}_{iness}$ is pruned and the remaining edges are kept in RCG. This is further explained here. First, for each pruned node $n \in \mathcal{N}_{iness}$ the edges connecting it to

- i. the two adjacent nodes on its lap are merged to form a single edge, and
- ii. all nodes on the adjacent laps are pruned.

After the inessential nodes and the corresponding edges are pruned, it is possible that there still exist some inessential edges between essential nodes. This is possible when some

essential nodes have edges connected to more than one essential nodes on an adjacent lap. Fig. 6b shows several examples of such cases. In the top-left corner, the top end node of the second lap from the left is connected to two essential nodes on the first lap, which are adjacent to the unknown area. Similar case arises with the top end node of the third lap. Another case arises with the bottom end node of the third lap from the right. This node is connected to three essential nodes in the right adjacent lap. Thus, further pruning of inessential edges is done. Fig. 6c shows examples of node and edge prunings. Fig. 6d shows the updated RCG after pruning.

The graph pruning results in the updated RCG, $\mathcal{G}_i = (\mathcal{N}_i, \mathcal{E}_i)$, with a sparse structure that reduces unnecessary computations and unwanted memory usage. It also enables distant (i.e., non-myopic) node selection as waypoints for smooth trajectory generation.

c) RCG Properties: The RCG has the following properties:

- P1. *Simple*: there is no more than one edge between any two nodes of RCG and no edge loops back on the same node.
- P2. *Planar*: RCG can be embedded on a plane such that no edges cross each other.
- P3. *Connected*: any node of RCG is reachable from any other node by traversing on the edges.
- P4. *Essential*: RCG is a minimum sufficient graph consisting of only essential nodes and edges.
- P5. *Sparse*: the nodes of RCG are distributed only close to the obstacles and unknown area.
- P6. *Scalable*: the size of RCG is $|\mathcal{N}| + |\mathcal{E}| \leq 4|\mathcal{N}| - 6$.

P1 and P2 directly result from the RCG expansion strategy, where the edges are only drawn from a node to connect the adjacent nodes on the same and side laps. P3 arises from the RCG expansion and pruning strategies, which build and maintain connections between all adjacent nodes on the same lap as well as connections to the ends nodes of adjacent laps, thus making every node reachable from every other node. P4 emerges from the fact that the nodes of RCG are formed from the frontier samples in the sampling front. After each pruning step only essential nodes and edges are left, and it is not possible to add any more frontier samples, thus making the RCG a minimum sufficient graph consisting of essential nodes and edges. Eventually the progressive sampling reaches the free space boundary by exploration until no more sampling front is found and no more frontier samples are created. Therefore, a fully formed RCG is a minimum sufficient graph for complete coverage. P5 appears because the RCG is created by using the frontier samples followed by the pruning process. P6 results from the Euler's Formula [50], which states that for a simple, connected, and planar graph $|\mathcal{E}| \leq 3|\mathcal{N}| - 6$.

4) Coverage Trajectory Generation: At each iteration $i \in \{0, \dots, m-1\}$, once the robot reaches the waypoint p_i , generates the sampling front $\mathcal{F}_i$, and updates the RCG to $\mathcal{G}_i = (\mathcal{N}_i, \mathcal{E}_i)$, the next step is to generate the next waypoint $p_{i+1} \in \mathcal{A}_f$. In order to generate p_{i+1} , one of the nodes $\hat{n}_{i+1} \in \mathcal{N}_i$ is selected as an intermediate goal node and its position is set as the next waypoint, such that $p_{i+1} = \text{pos}(\hat{n}_{i+1})$, where pos denotes the position of a node. The node selection procedure is described later. First, the node sequence is defined below.

Algorithm 1: Goal node selection at the current node

```

1 Function SelectGoalNode
2 Inputs to iteration  $i$ : current RCG  $\mathcal{G}_i$ ; current node  $\hat{n}_i$ ;  $\mathcal{N}^{\text{retreat}}$ .
3 if  $q(\hat{n}_i^L) = \text{Op}$  then
4    $\hat{n}_{i+1} = \hat{n}_i^L$ ; // if left node Op, go left
5 else if  $q(\hat{n}_i^U) = \text{Op}$  then
6    $\hat{n}_{i+1} = \hat{n}_i^U$ ; // elseif up node Op, go up
7 else if  $q(\hat{n}_i^D) = \text{Op}$  then
8    $\hat{n}_{i+1} = \hat{n}_i^D$ ; // elseif down node Op, go down
9 else if  $q(\hat{n}_i^R) = \text{Op}$  then
10   $\hat{n}_{i+1} = \hat{n}_i^R$ ; // elseif right node Op, go right
11 else // else there exists a dead-end at  $\hat{n}_i$ 
12   if  $\mathcal{N}^{\text{retreat}} \neq \emptyset$  then
13      $\hat{n}_{i+1} \leftarrow \text{EscapeDeadEnd}(\hat{n}_i, \mathcal{N}^{\text{retreat}})$ ;
14   else
15     Coverage is complete;
16   end
17 end
18  $p_{i+1} = \text{pos}(\hat{n}_{i+1})$ ;
19 return  $\hat{n}_{i+1}$  and  $p_{i+1}$ .

```

Algorithm 2: Updating the state of the current node

```

1 Function UpdateState
2 Inputs to iteration  $i$ : current node  $\hat{n}_i$ ; goal node  $\hat{n}_{i+1}$ ; current node
   state  $q(\hat{n}_i) = \text{Op}$ ;  $\mathcal{N}_i^{\text{Op}}$ ;  $\mathcal{N}_i^{\text{Cl}}$ .
3 if  $q(\hat{n}_i^U) = \text{Cl}$  or  $q(\hat{n}_i^D) = \text{Cl}$  then
4    $q(\hat{n}_i) = \text{Cl}$ ;
5    $\mathcal{N}_i^{\text{Op}} = \mathcal{N}_i^{\text{Op}} \setminus \hat{n}_i$ ;  $\mathcal{N}_i^{\text{Cl}} = \mathcal{N}_i^{\text{Cl}} \cup \hat{n}_i$ ;
6   if  $\hat{n}_{i+1} = \hat{n}_i^L$  then
7     if  $q(\hat{n}_i^L) = \text{Op}$  then
8       Create a link node above  $\hat{n}_i$  at a distance  $w$ ;
9     end
10    if  $q(\hat{n}_i^D) = \text{Op}$  then
11      Create a link node below  $\hat{n}_i$  at a distance  $w$ ;
12    end
13  end
14 end
15 return  $q_i(\hat{n}_i)$ ,  $\mathcal{N}_i^{\text{Op}}$ ,  $\mathcal{N}_i^{\text{Cl}}$ .

```

Definition III.10 (Node Sequence). The node sequence $\hat{\mathcal{N}} = \{\hat{n}_0, \hat{n}_1, \dots, \hat{n}_m\}$ is the sequence of intermediate goal nodes $\hat{n}_i \in \mathcal{N}$ for the robot $\mathcal{R}$ to form the coverage trajectory, where $\hat{n}_0$ and $\hat{n}_m$ are its start and end nodes, respectively.

The node sequence $\hat{\mathcal{N}} = \{\hat{n}_0, \hat{n}_1, \dots, \hat{n}_m\}$ has one-to-one correspondence to the waypoint sequence $P = \{p_0, p_1, \dots, p_m\}$, such that $p_i = \text{pos}(\hat{n}_i)$, $\forall i \in \{0, \dots, m\}$. The robot follows the waypoint sequence to form the coverage trajectory. In order to select the next goal node, the RCG needs to track the coverage progress by recording which nodes have been visited. Thus, the RCG assigns a state to each node, as follows.

Definition III.11 (State Encoding). Let $q : \mathcal{N} \rightarrow \{\text{Cl}, \text{Op}\}$ be the encoding that assigns each node $n \in \mathcal{N}$ of RCG, a state $q(n) \in \{\text{Cl}, \text{Op}\}$, where Cl (Op) $\equiv$ Closed (Open) implies that the node is visited (unvisited) by the robot.

At iteration $i \in \{0, \dots, m-1\}$, once the robot reaches $\hat{n}_i$ (i.e., waypoint p_i) and the RCG is updated to $\mathcal{G}_i$, the states of all new nodes in $\mathcal{G}_i$ are assigned to be Op . Let $\mathcal{N}_i^{\text{Op}} \subseteq \mathcal{N}_i$ and $\mathcal{N}_i^{\text{Cl}} \subseteq \mathcal{N}_i$ be the sets of *Open* and *Closed* nodes of $\mathcal{G}_i$, respectively, such that $\mathcal{N}_i^{\text{Op}} \cup \mathcal{N}_i^{\text{Cl}} = \mathcal{N}_i$. Once the node states are updated, the goal node $\hat{n}_{i+1}$ is selected as follows.

a) Goal Node Selection Strategy: Let $\hat{n}_i^L$, $\hat{n}_i^U$, $\hat{n}_i^D$ and $\hat{n}_i^R$ be the neighbors of the current node $\hat{n}_i$ on the left lap, the current lap upwards, the current lap downwards, and the right lap, respectively. Note: these directions are defined with respect to a fixed coordinate frame whose vertical axis is parallel to the laps. To obtain $\hat{n}_{i+1}$ (Alg. 1 lines 3-10) an *Open* node is searched from the neighbors of $\hat{n}_i$ and selected based on the priority order as follows: $\hat{n}_i^L \rightarrow \hat{n}_i^U \rightarrow \hat{n}_i^D \rightarrow \hat{n}_i^R$. In case there are more than one *Open* neighbors of $\hat{n}_i$ on the left or the right adjacent lap, then a random pick is done.

This simple node selection strategy generates the back and forth coverage pattern as follows. At any node, the robot first goes to the leftmost lap where an open node neighbor is available, then it moves on that lap until it finds another open node neighbor on the left or it reaches the end of the lap. If none of the above condition is true then it switches to the right lap and continues the coverage. In this manner, the robot performs the back and forth coverage lap by lap while shifting laps from left to right as they are covered.

Remark III.1. While moving on a lap, the robot's tasking device covers the region on both sides of the lap, such that the region between any two adjacent laps is covered if $w \leq 2r_c$. Thus, the sampling resolution w can be chosen smaller than $2r_c$ to generate slightly overlapping coverage pattern.

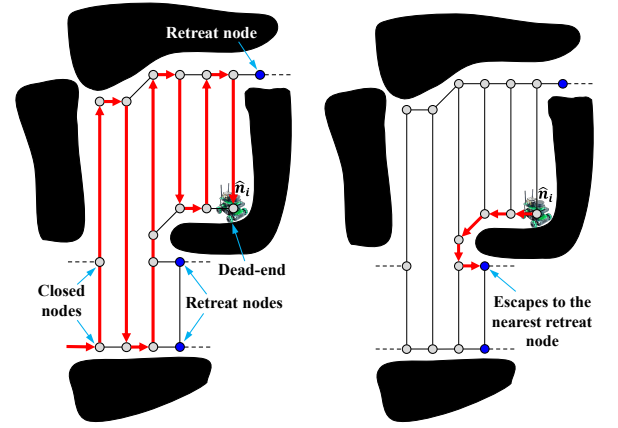
b) State Update Strategy: As soon as $\hat{n}_{i+1}$ is selected, the state of $\hat{n}_i$ is updated as $q(\hat{n}_i) = Cl$ (Alg. 2 lines 3-5). Only in case when both $\hat{n}_i^U \in \mathcal{N}_i^{Op}$ and $\hat{n}_i^D \in \mathcal{N}_i^{Op}$, the state of $\hat{n}_i$ is kept as $q(\hat{n}_i) = Op$. This is to avoid distortion of the back and forth trajectory by marking a node as *Closed* in the middle of *Open* nodes on a lap.

If the goal is selected on the left lap, i.e., $\hat{n}_{i+1} = \hat{n}_i^L$, and $q(\hat{n}_i) = Cl$, then it is further checked if either $\hat{n}_i^U \in \mathcal{N}_i^{Op}$ or $\hat{n}_i^D \in \mathcal{N}_i^{Op}$. If this is true then it means that the lap between $\hat{n}_i$ and the *Open* neighboring node above or below, if at a distance greater than w , is still uncovered. Since $q(\hat{n}_i) = Cl$, the robot would never travel on this lap. Thus, to cover this lap in the future, a new node, called link node, is created above or below $\hat{n}_i$ at a distance of w (Alg. 2 lines 6-13). The link nodes are pruned after they are covered and *Closed*.

c) Dead-end Escape Strategy: During navigation it is possible that the robot reaches a point where it is unable to find the next goal node because it is surrounded by only obstacles and covered regions. This is known as a dead-end situation. Fig. 8a shows an example of a dead-end.

Definition III.12 (Dead-end). A dead-end is said to occur at node $\hat{n}_i$ if $q(\hat{n}_i^L) = q(\hat{n}_i^U) = q(\hat{n}_i^D) = q(\hat{n}_i^R) = Cl$, which means that there is no *Open* node in the neighborhood $\mathcal{N}_b(\hat{n}_i)$ that could be selected as the next goal $\hat{n}_{i+1}$.

To escape from a dead-end and resume the back-and-forth coverage, an *Open* node needs to be found (Alg. 1 line 13). To select an open node 1) searching the entire graph is computationally inefficient and 2) randomly picking a node in uncovered region could distort the coverage trajectory. As such, C* uses the concept of retreat nodes as defined below.



(a) Robot reaches a dead-end. (b) Robot escapes from the dead-end to the nearest retreat node.

Fig. 8. Example of a robot escaping from a dead-end.

Definition III.13 (Retreat Node). A node $n \in \mathcal{N}_i^{Op}$ is defined as a retreat node if it is adjacent to the robot's trajectory.

The set $\mathcal{N}_{retreat} \subset \mathcal{N}_i^{Op}$ of retreat nodes is constantly updated during robot navigation. As shown in Fig. 8a, this is done by adding any *Open* node to $\mathcal{N}_{retreat}$ that is within a distance of $\sqrt{2}w$ from the robot and removing any *Closed* node. Note that there are always some retreat nodes in the list until coverage is complete because the area is connected. If the robot reaches a dead end at $\hat{n}_i$, then the closest retreat node is chosen as the next goal node $\hat{n}_{i+1}$. Subsequently, the robot moves to $\hat{n}_{i+1}$ on the shortest path obtained by the A* algorithm [39] and resumes the back-and-forth coverage of the remaining area. Fig. 8b shows an example of escaping from a dead-end to the nearest retreat node.

d) Coverage Holes Prevention Strategy: During navigation it is possible that the robot observes uncovered secluded regions that are surrounded by obstacles and covered regions. If bypassed, these regions form coverage holes, as shown in Fig. 9a, which then require long return trajectories to cover them later. This paper presents a proactive coverage hole prevention strategy that consists of: 1) detection of potential coverage holes and 2) coverage of such coverage holes by adaptation to TSP-based locally optimal coverage trajectories.

• **Detection of Potential Coverage Holes:** When the robot reaches node $\hat{n}_i$, it finds the next goal node $\hat{n}_{i+1}$. At this stage, it detects if there are any potential coverage holes near $\hat{n}_i$ that are surrounded by obstacles and covered regions. Let us index a coverage hole with $\alpha \in \mathbb{N}^+$ and denote the set of all nodes within this coverage hole with $\mathcal{N}_i^\alpha$.

The process of finding all such coverage holes near $\hat{n}_i$ is as follows. First, an *Open* node is picked up in the neighborhood of $\hat{n}_i$ and assigned an index $\alpha = 1$. Then, starting at this node, the neighboring *Open* nodes are recursively searched and labeled with index $\alpha = 1$. This is done using the floodfill function. The search continues to expand in all directions until reaching *Closed* nodes or the goal node $\hat{n}_{i+1}$. Once the search is complete then it is further checked if any *Open* node with index $\alpha = 1$ is adjacent to the unknown area. If true, then

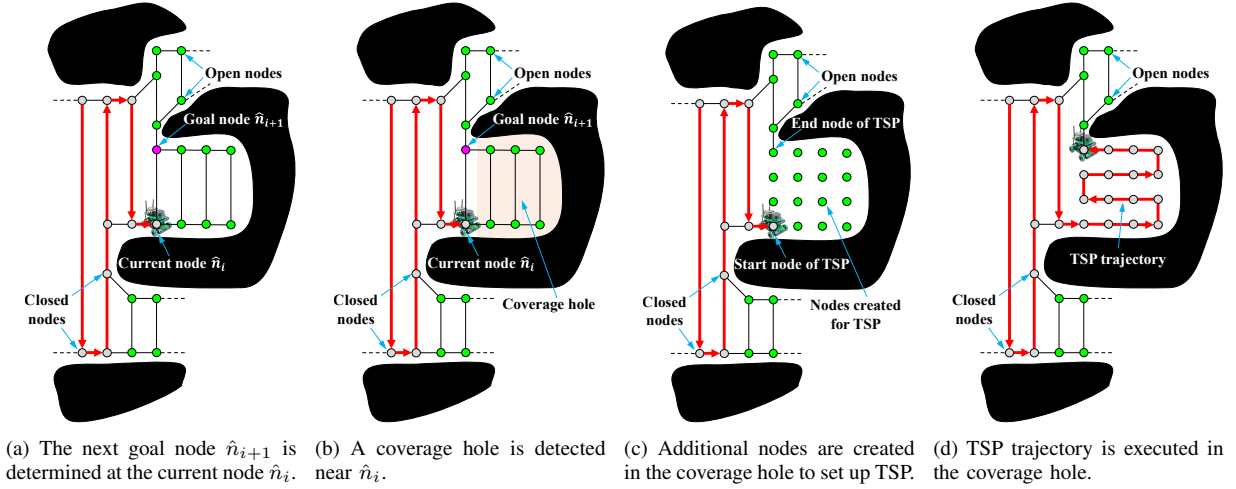


Fig. 9. An illustrative example of coverage hole detection and its coverage using the TSP trajectory.

Algorithm 3: Computation of the TSP trajectory

```

1 Function ComputeTSPTrajectory
2 Inputs to iteration  $i$ : Current node  $\hat{n}_i$ ; Goal node  $\hat{n}_{i+1}$ ;  $\mathcal{N}_i^{CH}$ .
3  $\overline{\mathcal{N}}_i^{CH} \leftarrow \text{AppendNodes}(\mathcal{N}_i^{CH}, \hat{n}_i, \hat{n}_{i+1})$ ;
4  $n_s^{TSP} = \hat{n}_i$ ;
5 if  $(\exists n \in \mathcal{N}_b(\hat{n}_{i+1}), \text{s.t. } q_i(n) = Op \text{ and } n \notin \overline{\mathcal{N}}_i^{CH}) \text{ or } \hat{n}_{i+1} \text{ is}$ 
   adjacent to unknown area then
6    $n_e^{TSP} = \hat{n}_{i+1}$ ;
7 else if  $\exists n \in \mathcal{N}_b(\hat{n}_i), \text{s.t. } q_i(n) = Op \text{ and } n \notin \overline{\mathcal{N}}_i^{CH} \text{ then}$ 
8    $n_e^{TSP} = \hat{n}_i$ ;
9 else
10   $n_e^{TSP}$  is not specified;
11 end
12  $\Lambda_i^{TSP} \leftarrow \text{SolveTSP}(\overline{\mathcal{N}}_i^{CH}, n_s^{TSP}, n_e^{TSP})$ ;
13 return  $\Lambda_i^{TSP}$ .

```

the set of nodes with index $\alpha = 1$ is disqualified to form a coverage hole. If not, then it forms a coverage hole. Fig. 9b shows an example where a coverage hole is detected near $\hat{n}_i$. The above search is repeated for any other unlabeled *Open* neighbor of $\hat{n}_i$ and labeling it and its *Open* neighbors with index $\alpha = 2$ until another coverage hole is detected and so on. Note that it is possible to have more than one coverage hole adjacent to $\hat{n}_i$. The search ends when all coverage holes near $\hat{n}_i$ are detected. Then $\mathcal{N}_i^{CH} = \bigcup_{\alpha} \mathcal{N}_i^{\alpha}$ denotes the set of all coverage holes around $\hat{n}_i$.

• **Set up TSP:** Once $\mathcal{N}_i^{CH}$ is discovered, it is covered using a locally optimal TSP trajectory Λ_i^{TSP} . To compute Λ_i^{TSP} , the TSP is set up as follows. Since RCG is built with only a sparse set of nodes, it is important to create additional nodes in $\mathcal{N}_i^{CH}$ to compute Λ_i^{TSP} . Thus, nodes are added on each lap inside a coverage hole with the inter-node distance of w . Fig. 9c shows an example of node addition inside a coverage hole to compute Λ_i^{TSP} . These nodes are then connected to form a fully connected sub-graph $\overline{\mathcal{G}}_i^{CH} = (\overline{\mathcal{N}}_i^{CH}, \overline{\mathcal{E}}_i^{CH})$. The node set $\overline{\mathcal{N}}_i^{CH}$ includes the current node $\hat{n}_i$, the goal node $\hat{n}_{i+1}$, the coverage hole nodes $\mathcal{N}_i^{CH}$ and the appended nodes. Each edge in $\overline{\mathcal{E}}_i^{CH}$ is assigned the A* [39] (shortest path) cost between its two end nodes.

The start node of the TSP trajectory is set to $n_s^{TSP} = \hat{n}_i$

(**Alg. 3 line 4**), while its end node n_e^{TSP} is determined as follows. If $\hat{n}_{i+1}$ is connected to an *Open* node which is not in the coverage hole then $n_e^{TSP} = \hat{n}_{i+1}$ (**Alg. 3 lines 5-6**). This allows for synergistic merging of Λ_i^{TSP} to the back-and-forth trajectory through $\hat{n}_{i+1}$. The above also applies when $\hat{n}_{i+1}$ is adjacent to unknown area; i.e., $\mathcal{B}(\hat{n}_{i+1}, w) \cap \mathcal{A}_u \neq \emptyset$. Else, if $\hat{n}_i$ is connected to an *Open* node which is not in the coverage hole then $n_e^{TSP} = \hat{n}_i$ (**Alg. 3 lines 7-8**). This allows for synergistic merging of Λ_i^{TSP} to the back-and-forth trajectory through $\hat{n}_i$. Note that $\hat{n}_i$ cannot be adjacent to unknown area because it is visited by the robot. If none of the above conditions are true then n_e^{TSP} is not specified.

• **Solve TSP:** Once the TSP is set up it is solved (**Alg. 3 line 12**) in an efficient manner to get the TSP trajectory Λ_i^{TSP} . For conditions 1 and 3 above (**Alg. 3 lines 5-6 and 9-10, respectively**), since $n_e^{TSP} \neq n_s^{TSP}$, a dummy node η is added to convert the problem into a closed-loop TSP that starts and ends at η . The edge cost between η and $\hat{n}_i$ is set as $u_{\eta, \hat{n}_i} = 0$. For condition 1, the edge cost between η and $\hat{n}_{i+1}$ is set as $u_{\eta, \hat{n}_{i+1}} = 0$, while for condition 3, it is set as $u_{\eta, \hat{n}_{i+1}} = \infty$. The edge costs between η and all the remaining nodes is set as $u_{\eta, n} = \infty, \forall n \in \overline{\mathcal{N}}_i^{CH} \setminus \{\hat{n}_i, \hat{n}_{i+1}\}$. Note that η is removed once the optimized node sequence Λ_i^{TSP} is obtained. For condition 2 (**Alg. 3 lines 7-8**), since $n_e^{TSP} = n_s^{TSP}$, the TSP is closed-loop; thus no dummy node is needed.

A heuristic approach is used to solve for TSP as it is NP hard [51]. First, the nearest neighbor algorithm [51] is used to obtain an initial node sequence. Then, the 2-opt algorithm [51] is used to improve this initial solution and obtain the optimized node sequence. Fig. 9d shows an example of the TSP trajectory executed inside a coverage hole.

IV. ALGORITHM ANALYSIS

Algorithm 4 shows the iterative steps of C*. The complexity of C* is evaluated for each iteration i when the robot reaches a node $\hat{n}_i$. Since the environment mapping is done continuously during robot navigation, the complexity of C* is evaluated for the steps of progressive sampling, RCG expansion, RCG

Algorithm 4: C* execution

```

1 Inputs to iteration  $i$ : starting location  $p_{i-1}$ ; existing RCG  $\mathcal{G}_{i-1}$ ;
  next waypoint  $p_i$ ; goal node  $\hat{n}_i$ 
2 while  $\mathcal{N}_{i-1}^{Op} \neq \emptyset$  do
3    $\mathcal{A}_{d,i} \leftarrow \text{NavigateSense}(p_{i-1}, p_i)$ ; // navigates from
     $p_{i-1}$  to  $p_i$  and discovers the environment
4    $\mathcal{F}_i \leftarrow \text{CreateSamplingFront}(\mathcal{A}_{d,i})$ ; // creates a
    sampling front upon reaching  $p_i$ 
5    $\mathcal{S}_i \leftarrow \text{GenerateFrontierSamples}(\mathcal{F}_i)$ ; // Generates
    frontier samples on each lap in  $\mathcal{F}_i$ 
6    $\mathcal{G}'_{i-1} \leftarrow \text{ExpandRCG}(\mathcal{G}_{i-1}, \mathcal{S}_i)$ ; // assigns  $\mathcal{S}_i$  as
     $\mathcal{N}_{new}$  and adds connecting edges to  $\mathcal{G}_{i-1}$ 
7    $\mathcal{G}_i \leftarrow \text{PruneRCG}(\mathcal{G}'_{i-1})$ ; // generates updated RCG
    by pruning inessential nodes and edges
8    $(\hat{n}_{i+1}, p_{i+1}) \leftarrow \text{SelectGoalNode}(\mathcal{G}_i, \hat{n}_i)$ ; // selects
    goal node, if a dead-end exists then uses
    the dead-end escape function as in Alg 1
9    $(q_i(\hat{n}_i), \mathcal{N}_i^{Op}, \mathcal{N}_i^{Cl}) \leftarrow \text{UpdateState}(q_i(\hat{n}_i), \mathcal{N}_i^{Op}, \mathcal{N}_i^{Cl})$ ;
    // updates the state of  $\hat{n}_i$  as in Alg 2
10   $\mathcal{N}_i^{CH} \leftarrow \text{DetectCoverageHoles}(\mathcal{G}_i, \hat{n}_i, \hat{n}_{i+1})$ ;
11  if  $\mathcal{N}_i^{CH} \neq \emptyset$  then
    // coverage hole exists
12     $\Lambda_i^{TSP} \leftarrow \text{ComputeTSPTrajectory}(\mathcal{N}_i^{CH}, \hat{n}_i, \hat{n}_{i+1})$ ;
     $n_e^{TSP} \leftarrow \text{ExecuteTSPTrajectory}(\Lambda_i^{TSP})$ ;
13     $\hat{n}_i = n_e^{TSP}$ ;  $\hat{n}_{i+1} = n_e^{TSP}$ ;
14  end
15   $i = i + 1$ ;
16 end

```

pruning, goal node selection and RCG state updating. The complexities of dead-end escape and coverage holes prevention strategies are evaluated separately.

Lemma IV.1. *The complexity of sampling is $O(|\mathcal{S}_{\mathcal{F}_i}|)$.*

Proof. Let $\mathcal{S}_{\mathcal{F}_i}$ be the set of all possible samples that could be placed uniformly in the sampling front $\mathcal{F}_i$ with the sampling resolution of w . In order to pick the set of frontier samples $\mathcal{S}_i \subseteq \mathcal{S}_{\mathcal{F}_i}$, it is required to check the ball $\mathcal{B}(s, w)$ around each potential sample $s \in \mathcal{S}_{\mathcal{F}_i}$ to verify if s is adjacent to unknown area and/or obstacle. Thus, the complexity of progressive sampling is $O(|\mathcal{S}_{\mathcal{F}_i}|)$. $\square$

Remark IV.1. $|\mathcal{S}_{\mathcal{F}_i}| < \lfloor \frac{2r_d}{w} \rfloor \lfloor \frac{\ell_i^{max}}{w} \rfloor$, where $\lfloor \cdot \rfloor$ is the floor operation, $\lfloor \frac{2r_d}{w} \rfloor$ is the maximum number of laps in a sampling front, and $\lfloor \frac{\ell_i^{max}}{w} \rfloor$ is the maximum number of samples on the longest lap in $\mathcal{F}_i$ with length $\ell_i^{max} \in \mathbb{R}^+$. Note that since $\mathcal{F}_i$ is generated by sensing during navigation from p_{i-1} to p_i , ℓ_i^{max} could be greater than $2r_d$.

Lemma IV.2. *The complexity of RCG expansion is $O(|\mathcal{S}_i|)$.*

Proof. First, the new nodes $\mathcal{N}_{new}$ of RCG are created from the frontier samples $\mathcal{S}_i$ and added to its node set $\mathcal{N}_{i-1}$. This has the complexity of $O(|\mathcal{N}_{new}|)$. Then, the new edges are created and added to the edge set $\mathcal{E}_{i-1}$ by connecting the new nodes to each other and to the neighboring nodes of existing RCG. For each new node, new edges are drawn to connect it to at most 2 adjacent nodes on the same lap and at most 3 nodes on each adjacent lap that are within a distance of $\sqrt{2}w$. This has the complexity of $O(8(|\mathcal{N}_{new}|))$. Thus, the complexity of RCG expansion is $O(9|\mathcal{N}_{new}|) = O(|\mathcal{N}_{new}|) = O(|\mathcal{S}_i|)$. $\square$

Lemma IV.3. *The complexity of node pruning is $O(|\mathcal{S}_i|)$.*

Proof. For each node $n \in \{\mathcal{N}_{new} \cup \mathcal{N}_{\partial \mathcal{F}_i}\}$, we follow Def. III.8 to check if its essential or inessential. Condition 1 of Def. III.8 checks around node n to verify if it is adjacent to the unknown area. Condition 2 checks if n is an end node of its lap. Condition 3 checks at most 6 neighbors of n on adjacent laps to verify if it is connected to an end node n_x of the adjacent lap. Condition 3a checks if n_x has other neighbor(s) on n 's lap. Condition 3b checks if the other neighbors (at most 2) of n_x on n 's lap are non-end nodes. Condition 3b further checks if the edge $e(n, n_x)$ connecting n and n_x is closest to an obstacle or the unknown area. To check this, k intermediate points are created on the edge and their neighborhoods are checked to compute the distance to the nearest obstacle. Based on the above procedure, the set of inessential nodes $\mathcal{N}_{iness}$ is determined and pruned from RCG. Thus, a fixed number of computations are done for each node $n \in \{\mathcal{N}_{new} \cup \mathcal{N}_{\partial \mathcal{F}_i}\}$. Thus, the node pruning has a complexity of $O(|\mathcal{N}_{new}| + |\mathcal{N}_{\partial \mathcal{F}_i}|)$. Typically, $|\mathcal{N}_{\partial \mathcal{F}_i}| \leq |\mathcal{N}_{new}| = |\mathcal{S}_i|$, thus, the node pruning complexity is $\sim O(|\mathcal{S}_i|)$. $\square$

Lemma IV.4. *The complexity of edge pruning is $O(|\mathcal{S}_i|)$.*

Proof. For each edge $e(n_x, n_y) \in \{\mathcal{E}_{new} \cup \mathcal{E}_{\partial \mathcal{F}_i}\}$, we follow Def. III.9 to check if its essential or inessential. First, for each pruned node $n \in \mathcal{N}_{iness}$, the edges connecting it to the two adjacent nodes on the same lap are merged and the edges connecting it to the nodes on adjacent laps are pruned. This satisfies Conditions 1 and 2a of Def. III.9. Then, Condition 2b-i checks if n_x has no other neighbor on n_y 's lap. Condition 2b-ii checks if all other neighbors on n_y 's lap are non-end nodes. It further checks if the edge $e(n_x, n_y)$ is closest to an obstacle or the unknown area. To check this, k intermediate points are created on $e(n_x, n_y)$ and their neighborhoods are checked to compute the distance to the nearest obstacle. Based on the above procedure, the inessential edges are pruned from RCG. Thus, a fixed number of computations are done for each edge $e(n_x, n_y) \in \{\mathcal{E}_{new} \cup \mathcal{E}_{\partial \mathcal{F}_i}\}$. Thus, the edge pruning has a complexity of $O(|\mathcal{E}_{new}| + |\mathcal{E}_{\partial \mathcal{F}_i}|)$. Typically, $|\mathcal{E}_{\partial \mathcal{F}_i}| \leq |\mathcal{E}_{new}| \leq 3(|\mathcal{N}_{new}| + |\mathcal{N}_{\partial \mathcal{F}_i}|) - 6$, and $|\mathcal{N}_{\partial \mathcal{F}_i}| \leq |\mathcal{N}_{new}| = |\mathcal{S}_i|$, thus, the edge pruning complexity is $\sim O(|\mathcal{S}_i|)$. $\square$

Lemma IV.5. *The complexity of goal node selection is $O(1)$.*

Proof. To select the goal node at $\hat{n}_i$, an open node is searched in the neighborhood of $\hat{n}_i$ based on the priority order of 1) $\hat{n}_i^L$, 2) $\hat{n}_i^U$, 3) $\hat{n}_i^D$, and 4) $\hat{n}_i^R$. This has a complexity of $O(1)$. $\square$

Lemma IV.6. *The complexity of node state updating is $O(1)$.*

Proof. Once the goal node $\hat{n}_{i+1}$ is selected, the algorithm checks if the nodes adjacent to the current node $\hat{n}_i$ on the same lap are both *Open*. If yes, then the state of $\hat{n}_i$ is kept as *Open*; otherwise, it is updated to *Closed*. Further, if $\hat{n}_{i+1}$ is selected on the left lap and the state of $\hat{n}_i$ is *Closed*, then it is checked if either $\hat{n}_i^U$ or $\hat{n}_i^D$ is farther than w is *Open*. If this is true, then a link node is created above or below $\hat{n}_i$ at a distance of w . The above process requires a fixed number of operations, thus the complexity of state updating is $O(1)$. $\square$

Theorem IV.1. *The complexity of C^* is $O(|\mathcal{S}_{\mathcal{F}_i}|)$.*

Proof. From Lemmas IV.1-IV.6, the complexity of C^* is $O(|\mathcal{S}_{\mathcal{F}_i}| + 3|\mathcal{S}_i| + 1 + 1)$. Since $\mathcal{S}_i \subseteq \mathcal{S}_{\mathcal{F}_i}$, the overall complexity of C^* is $O(|\mathcal{S}_{\mathcal{F}_i}|)$. Note that from Remark IV.1, the complexity increases if the sampling resolution $w \in \mathbb{R}^+$ is decreased. $\square$

Since the dead-ends and coverage holes do not occur at all iterations, the complexities of dead-end escape and coverage hole prevention strategies are evaluated separately.

Lemma IV.7. *The complexity of dead-end escape strategy is $O(|\mathcal{N}_i| \log |\mathcal{N}_i|)$.*

Proof. To escape from a dead-end, the nearest retreat node with the shortest path is selected from $\mathcal{N}_{retreat}$. Thus, the path length to each retreat node is calculated using A* and then the node with the shortest path length is selected. Since A* has a complexity of $O(|\mathcal{E}_i| \log |\mathcal{N}_i|)$, the overall complexity of this process is $O(|\mathcal{N}_{retreat}|(|\mathcal{E}_i| \log |\mathcal{N}_i| + 1))$. Since $|\mathcal{E}_i| \leq 3|\mathcal{N}_i| - 6$ and usually $|\mathcal{N}_{retreat}| \ll |\mathcal{N}_i|$, the complexity of dead-end escape strategy is $O(|\mathcal{N}_i| \log |\mathcal{N}_i|)$. $\square$

Lemma IV.8. *The complexity of coverage hole prevention strategy is $O(|\mathcal{N}_i|)$.*

Proof. To find any coverage holes around node $\hat{n}_i$, a recursive search is conducted for Open nodes in the neighborhood. This has the worst case complexity of $O(|\mathcal{N}_i|)$; however, in practice it is significantly smaller. If a coverage hole is detected, then a fully connected sub-graph $\bar{\mathcal{G}}_i^{CH} = (\bar{\mathcal{N}}_i^{CH}, \bar{\mathcal{E}}_i^{CH})$ is formed in the coverage hole and a TSP is formulated. An initial solution of TSP is obtained using a nearest-neighbor based heuristic approach which has a complexity of $O(|\bar{\mathcal{N}}_i^{CH}|^2)$ [51]. Then, the 2-Opt greedy algorithm is applied to improve the solution which has a complexity of $O(|\bar{\mathcal{N}}_i^{CH}|)$ [51]. Typically, $|\bar{\mathcal{N}}_i^{CH}| \ll |\mathcal{N}_i|$, thus the complexity of coverage hole prevention is $O(|\mathcal{N}_i|)$. $\square$

Theorem IV.2 (Space Complexity). *The space complexity of the C^* algorithm is $O(|\mathcal{N}_i|)$.*

Proof. The space complexity is the amount of memory used to store the RCG $\mathcal{G}_i(\mathcal{N}_i, \mathcal{E}_i)$. The size of $\mathcal{G}_i$ is $|\mathcal{N}_i| + |\mathcal{E}_i|$. Since $|\mathcal{E}_i| \leq 3|\mathcal{N}_i| - 6$, the space complexity is $O(|\mathcal{N}_i|)$. $\square$

Theorem IV.3 (Complete Coverage). *C^* achieves complete coverage of the obstacle-free space.*

Proof. The proof follows from the RCG construction, goal node selection and dead-end escape strategies.

- First, the connectivity of obstacle-free space and progressive sampling at all the frontier nodes adjacent to the unknown area imply that RCG will incrementally grow and eventually spread in all of the free space.
- Second, RCG is a minimum sufficient graph (P4) constructed from the frontier samples. This implies that the nodes of a fully grown RCG will lie within a ball of radius w to the obstacles or the boundary of the free space. Thus, a fully grown RCG will wrap tightly around the boundary of the obstacle-free space.

- Third, the goal node selection strategy in Section III-B4a ensures that the robot performs the back-and-forth coverage lap by lap while shifting laps from left to right as they are covered. From Remark III.1 it is guaranteed that the region between any two adjacent laps is covered if $w \leq 2r_c$. By connectivity of RCG (P3) it is further guaranteed that each lap is covered. The TSP-based coverage of potential coverage holes further optimizes the coverage trajectory in local regions surrounded by obstacles before resuming the lap by lap coverage.
- Finally, the coverage continues until all nodes of RCG are visited or a dead-end is reached. The robot escapes from a dead-end to the closest retreat node and resumes the back-and-forth coverage. If there is no retreat node available, it implies that the complete coverage is achieved.

The above procedures guarantee that C^* achieves complete coverage of the free space. $\square$

V. RESULTS AND DISCUSSION

This section presents the results of testing and validation of C^* by 1) high-fidelity simulations, 2) real-experiments on autonomous robots in a laboratory setting, and 3) applications on two different CPP problems considering a) energy-constrained robots and b) multi-robot teams.

A. Validation on a Simulation Platform

The performance of C^* is first evaluated by extensive high-fidelity simulations on a multitude of complex scenarios. A robotic simulation platform, called Player/Stage [52] is used for simulations, where Player is a robot server that contains models of different types of robots and sensors, while Stage is a highly configurable simulator of a variety of robotic operations. Several complex scenarios of dimensions $50m \times 50m$ are randomly generated with different obstacle layouts to test and validate the performance of C^* . These scenarios are considered as unknown to the robot at the beginning of the CPP operation. In order to detect and map obstacles during navigation, the autonomous robot is equipped with a laser sensor that has the FOV with a span of 360° and a detection range of $15m$. The robot has motion constraints including the top speed of $0.5m/s$, max acceleration of $0.5m/s^2$, and the minimum turning radius of $0.04m$. The robot dynamically discovers the environment during navigation and generates adaptive coverage trajectory until complete coverage is achieved.

1) Baseline Algorithms: Seven baseline algorithms are selected for performance comparison including: PPCPP [13], ϵ^* [3], BA* [22], NNCP [23], BM [24], BSA [25], and FSSTC [26]. Table I compares the key features of C^* with the baseline algorithms. All algorithms were deployed in C++ and run on a computer with 2.60 GHz processor and 32 GB RAM. The sampling resolution for C^* and the grid resolution for the baseline algorithms are set as $1m$.

2) Performance Metrics: The following performance metrics are considered for comparative evaluation:

- Coverage Time (CT): the total time of the coverage trajectory to achieve complete coverage.